A Traffic Light Model to Save Energy

A Science Research Paper Presented to

Newton Country Day School Science and Engineering Fair

March 1, 2023

Ludovica Melodia

Mr. Molnar

Grade 10

Table of Contents

Acknowledgments…….....2

Abstract.…….3

Introduction…….……4

Experimental Design……...…….....8

Data and Results……..17

Discussions and Conclusions……...……22

Bibliography……..……...…25

Acknowledgements

I would like to thank the Newton Country Day science department for giving me this opportunity to undertake this project. I would especially like to thank Mr. Molnar, my mentor throughout this process, for helping me overcome all of the obstacles which I was faced with as well as keeping me organized. I would also like to thank my parents for supporting me throughout this process; my father for helping me fix errors in my code and my mother for doing it all, whether driving me to pick up materials or helping me memorize my presentation. Finally, I want to thank my friends Lily and Kevin for helping me run my trials. I would not have been able to perform my testing process correctly without them.

Abstract

There are millions of traffic lights throughout the world that are always on, constantly consuming energy. The energy that is used from all of these traffic lights impacts the environment. Energy waste is a prominent issue in today's world, which is why it should be an important societal goal to work towards energy efficiency.

Internet of Things (IoT) is a concept in which objects in our environment can communicate with each other to create more efficient systems. I created an IoT model for a traffic light which only turns on when it can sense a car in range. Unlike other already existing sensor operated traffic lights, the light in my model communicates with cars using Bluetooth, enabling it to have an accurate sense of where each nearby vehicle is. This model, which includes two traffic lights to represent an intersection, operates on a fixed rotation which begins whenever a car can be sensed in range. I used an Arduino board with a built-in Bluetooth module to control the lights, and a smartphone with Bluetooth capability to represent the car in motion.

I tested the model in 10 trials in which I recorded the distances at which the phone and model connected and disconnected. The trials demonstrated that the phone was always able to connect when approaching the light and disconnect while receding, and that the lights always turned on and off in correspondence with the successful connections. One pattern that can be observed from the trials is that the average distance for disconnecting was greater than the average distance for establishing connections.

Introduction

About two thirds of the energy that is produced each year in the US is wasted (Webber and Rhodes, 2017). The exaggerated use of energy in today's society is a large contributor to the pollution and overall destruction of the Earth. While the use of environmentally friendly ways to create energy is increasing, the three most common energy sources are natural gas, coal, and nuclear power. Natural gas and coal are nonrenewable resources which our current society consumes at alarmingly rapid rates, while nuclear power comes with many risks, and produces radioactive waste. The U.S. electricity system has a tremendous environmental impact - it causes greenhouse gas emissions, excessive use of water, water pollution, and generation of solid waste. In addition, the production of electricity takes up vast amounts of land, and harms the ecosystems of this land (EPA, 2022). It is important to develop solutions to make everyday life more energy efficient, in order to avoid all of these effects which are harming the Earth. One area in which energy usage could be reduced concerns traffic lights.

Traffic lights are on all the time, a constant consummation of energy. Even in certain times or areas in which there are usually no cars, traffic lights cannot be turned off in the eventuality that a car does come through. Each year, an estimated 3 billion kilowatts of energy are consumed on traffic lights (Fama Traffic, 2020). Additionally, idling cars in traffic account for an estimated 30 million tons of carbon dioxide annually. Carbon dioxide is a greenhouse gas, and this immense amount of greenhouse emissions furthers the environmental impact of inefficient lights. Traffic lights consume unnecessary amounts of energy by remaining on even in the late hours of the night, and in deserted streets.

Major traffic lights, in cities and areas with dense traffic, operate on a timer. The color of the lights cycles through red, yellow and green lights in timed intervals in order to keep the flow

of cars even from all directions. These traffic lights run on pre-programmed systems and do not have sensors, and therefore do not run based on the number and movement of cars that are in the area at any given moment. Traffic lights in more minor areas often have sensors which tell them when to change the signal for an approaching car. Examples of sensors that are used in these types of lights are lasers, video cameras, or “inductive loop systems” - wires in the road that can sense when a car is present. In these smaller areas, the lights on the main road tend to stay on green, and when the sensors sense a car on the side road, the lights change to allow it to pass. These sensors do not always consider cyclists and pedestrians, and the weather can interfere with them (Wonderopolis, n.d.).

Right now, only three percent of all traffic lights in the United States are considered “smart”(Austin 2019). While some minor traffic lights can sense the presence of a car, none are able to communicate with and receive signals from cars. In order to do this, they would require wireless communication technology. Wireless communication systems are used for different objects, such as televisions with remotes and GPS systems, to communicate without a physical medium. Instead of using a cable to carry the signals like in wired communication, wireless communication uses Electromagnetic (EM) Waves - typically Microwave Rays and Radio Waves. These systems function with three main components: a transmitter, a channel, and a receiver. The transmitter is the source of the signal, and where the signal is encoded, encrypted, modulated and multiplexed in order to be transmitted through an antenna. The channel, or medium of transmission of the signal, is where the signal travels. Wired communication functions with a guided medium, such as cables, for the signal to travel on. Wireless communication, however, uses an unguided medium - it is called this way because the signal propagates through space without any guidance. These waves do not have a distinct shape or

path, which is why there cannot be a guide made for them. The final component is the receiver. The receiver basically has to take the code from the channel and transform it back into what it was before it went through the transmitter. The signal must go through Demodulation, Channel Decoding, Decryption and Source Decoding. The final step, the source decoder, is what brings the signal back to the original message (Electronics hub, 2021).

One example of wireless communication technology is Bluetooth. Bluetooth uses weaker radio waves than cellular and WiFi communications, because it only works for short distances (usually less than 30 feet/9 meters). Bluetooth enabled devices are able to automatically detect each other and connect whenever they are in range of each other. When two devices connect via Bluetooth, they choose an available frequency at random, and connect through it. Bluetooth could be used in the street because it does not require WiFi or internet connection to work - it is its own communication system. All that is needed for a Bluetooth connection is two devices with a Bluetooth chip. Bluetooth uses 79 different radio frequencies, at approximately 2.4 gigahertz of frequency (Raikar, 2022). Bluetooth Low Energy (BLE) is a Bluetooth radio designed for very low power operation. It uses 40 channels in the 2.4GHz frequency band, and has a greater range than regular Bluetooth, reaching up to 100 meters. BLE has a lower cost and power consumption than regular Bluetooth. It also includes features that enable one device to determine the presence, distance, and direction of another device (“Bluetooth Technology, nd).

There are already working solutions to reduce energy waste from traffic lights. One of these is a system that is being developed and put into effect in some areas for “smart traffic lights.” In this system, created by a company called Surtrac, Artificial Intelligence (AI) traffic lights can communicate with each other. These lights use sensors and cameras to gather real time information about what is happening on the street and then send this information to one another

to create the most efficient traffic patterns. With this system, the traffic signals in any given place can consider specific scenarios. For example, if a person in a wheelchair is crossing the street, a camera could sense this and let the nearby lights at its intersection know that they will need to stay on for some time longer. The company found that with this network of smart traffic lights, travel time decreased by 25-26% (Austin, 2019). Another model was developed for a smart traffic light system in which traffic lights can sense cyclists through signals from their cell phones (Anagnostopoulos et al., 2016). There is also a campaign to replace the lights in traffic lights with LEDs, which consume less energy (Fama Traffic, 2020). LED bulbs use between 6 and 22 watts, while incandescent bulbs require 135 watts. In a study that tested traffic lights with LED bulbs versus incandescent bulbs in two similar intersections over a period of four months, it was found that the LEDs consumed an average of 111 kWh per month, while the incandescent lights consumed an average of 1203 kWh per month. Using LEDs also has financial benefits - while the bulbs cost more, the significantly smaller energy consumption results in a much smaller energy bill (anonymous, n.d.).

Inefficient traffic lights waste not only energy, but also time. About 20 percent of all driving time is spent waiting at red lights, where the average American will spend a total of 6 months of their life (Taylor, 2020). In sum, the current system for traffic lights is outdated and wasteful.

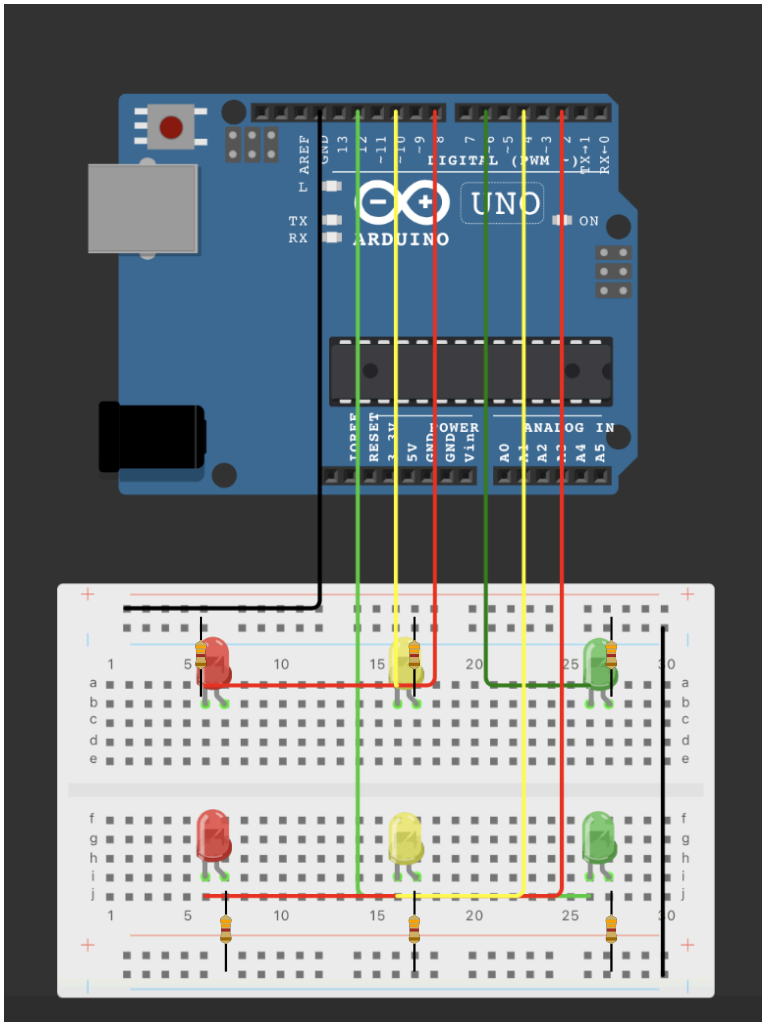
Experimental Design

Materials:

- Arduino Uno WiFi Rev2 circuit board
- 2 green, 2 yellow, 2 red, 2 black jumper wires
- 6 330Ω resistors
- 2 red, 2 yellow, 2 green 10 mm LED lights
- 9V alkaline battery
- 9V battery clip connector

Tools:

- Alcatel TETRA phone, Android version 8.1.0
- Computer with Arduino IDE software, version 2.0.2



Building process

- Take the 6 colored jumper wires and the Arduino board
- Connect one red wire from pin 2 on the Arduino to 6j on the breadboard
- Connect the second red wire from pin 8 on the Arduino to 6a on the breadboard
- Connect one yellow wire from pin 4 on the Arduino to 16j on the breadboard
- Connect the second yellow wire from pin 10 on the Arduino to 16a on the breadboard
- Connect one green wire from pin 6 on the Arduino to 26j on the breadboard
- Connect the second green wire from pin 12 on the Arduino to 26a on the breadboard
- Take the two black jumper wires

- Connect one black wire from the GND pin on the Arduino to the topmost negative pin on the breadboard
- Connect the second black wire from one negative pin on the bottom breadboard to one on the other side
- Take the 6 resistors and put each with one side in the breadboard pin directly below a colored wire, and the other side in a negative pin
- Take the 6 LED lights
- Put one red light with the longer side in pin 6b and the shorter side in pin 7b
- Put the other red light with the longer side in pin 6i and the shorter side in pin 7i
- Put one yellow light with the longer side in pin 16b and the shorter side in pin 17b
- Put the other yellow light with the longer side in pin 16i and the shorter side in pin 17i
- Put one green light with the longer side in pin 26b and the shorter side in pin 27b
- Put the other green light with the longer side in pin 27i and the shorter side in pin 27i
- Connect the battery to the clip connector and plug it into the Arduino board
- Connect the Arduino board to a computer with a USB cable
- Upload the code onto the Arduino board using the Arduino IDE software:

```
#include <ArduinoBLE.h>

  // Variables will change:
  int red = LOW;  // state used of red LED initially set the off
  int yellow = LOW; // state used of yellow LED initially set the off
  int green = LOW; // state used of green LED initially set the off
  int red2 = LOW; // state used of red2 LED initially set the off
```

```
  int yellow2 = LOW; // state used of yellow2 LED initially set the
off
  int green2 = LOW; // state used of green2 LED initially set the off

  unsigned long previousMillis = 0;  // will store last time LED state
was updated

  const long interval = 6000;  // interval for red light (in
milliseconds)
  const long greenduration = 4000; // interval for green light
  const long yellowduration = interval - greenduration; // interval
for yellow light

void setup() {
  Serial.begin(9600);
  while (!Serial);


  pinMode(LED_BUILTIN, OUTPUT);           // set LED's pin to output
mode
  pinMode(2, OUTPUT);                     // initialize digital pin 2
as an output (red)
  pinMode(4, OUTPUT);                     // initialize digital pin 4
as an output (yellow)
  pinMode(6, OUTPUT);                     // initialize digital pin 6
as an output (green)
```

```
  pinMode(8, OUTPUT);                       // initialize digital pin 8
as an output (red2)
  pinMode(10, OUTPUT);                       // initialize digital pin
10 as an output (yellow2)
  pinMode(12, OUTPUT);                      // initialize digital pin 10
as an output (green2)

  digitalWrite(LED_BUILTIN, LOW);           // when the central
disconnects, turn off the LED


  // begin initialization
  if (!BLE.begin()) {
    Serial.println("starting Bluetooth® Low Energy failed!");

    while (1);
  }

  // set advertised local name and service UUID:
  BLE.setLocalName("Arduino Uno Wifi Rev2");

  // start advertising
  BLE.advertise();
  Serial.println("BLE Arduino Uno Wifi Rev2 Peripheral");
}
```

```
void loop() {
  // listen for Bluetooth® Low Energy peripherals to connect:
  BLEDevice central = BLE.central();



  // if a central is connected to peripheral:
  if (central) {
    Serial.print("Connected to central: ");
    Serial.println(central.address());          // print the central's
MAC address:
    digitalWrite(LED_BUILTIN, HIGH);            // turn on the LED to
indicate the connection

  // check to see if it's time to blink the LED; that is, if the
difference
  // between the current time and last time you blinked the LED is
bigger than
  // the interval at which you want to blink the LED.

    // while the central is still connected to peripheral:
    while (central.connected()) {
      unsigned long currentMillis = millis();
      if (currentMillis - previousMillis <= greenduration) {
        red = HIGH;
        green = LOW;
        yellow = LOW;
```

```
        red2 = LOW;
        green2 = HIGH;
        yellow2 = LOW;
      }
      else if (currentMillis - previousMillis <= interval) {
              red = HIGH;
              green = LOW;
              yellow = LOW;
              red2 = LOW;
              green2 = LOW;
              yellow2 = HIGH;
            } else if (currentMillis - previousMillis <= ( interval +
greenduration)) {
                red = LOW;
                green = HIGH;
                yellow = LOW;
                red2 = HIGH;
                green2 = LOW;
                yellow2 = LOW;
              } else {
                  red = LOW;
                  green = LOW;
                  yellow = HIGH;
                  red2 = HIGH;
                  green2 = LOW;
                  yellow2 = LOW;
```

```
                    if (currentMillis - previousMillis == 2*interval){
                        previousMillis = currentMillis;
                      }
                }


        // set the LED with the state (red, yellow or green) of the
variable:
        digitalWrite(2, red);
        digitalWrite(4, yellow);
        digitalWrite(6, green);
        digitalWrite(8, red2);
        digitalWrite(10, yellow2);
        digitalWrite(12, green2);
      }
      //}

    // when the central disconnects, print it out:
    Serial.print(F("Disconnected from central: "));
    Serial.println(central.address());
    digitalWrite(LED_BUILTIN, LOW);
    digitalWrite(2, LOW); // turn off red light when disconnected
    digitalWrite(4, LOW); // turn off yellow light when disconnected
    digitalWrite(6, LOW); // turn off green light when disconnected
    digitalWrite(8, LOW); // turn off red2 light when disconnected
    digitalWrite(10, LOW); // turn off yellow2 light when disconnected
```

```
    digitalWrite(12, LOW); // turn off green2 light when disconnected


  }
}
```

Modification Process

The initial design was a simple circuit with two LED lights, two resistors, and three jumper wires. It consisted of a yellow and red light that blinked alternatingly when the board sensed a bluetooth signal. I used the "delay" command in my code in order for each light to go on for one second and off for one second, alternating with each other. When I connected my phone to the board via BLE, the red and yellow lights flashed in alternation. This model with two alternating lights was the base for one with six lights, so once I was able to make it work with the two lights I added the other four lights to the board and modified my code to include them. In this way, three lights represented one traffic light and the other three represented another, so the model could represent an intersection with two traffic lights alternating their lights in sync. After adding the lights, I had to figure out a rotation that made sense in order for both lights to alternate between red, yellow and green in sync with each other. I made this work by turning on each green light for 4 seconds, then yellow for 2 seconds, then red for 6. In this way, one light would go between yellow and green, allowing cars to pass, while the other stayed red. Upon trying to connect the circuit to bluetooth, I noticed that the lights would freeze into the alternation after the first time being turned on and no longer respond to Bluetooth connections or disconnections. This was a problem because the goal of the experiment was for the lights to turn off when they no longer sensed a connection (representing a nearby car). I discovered that this block occurred due to the "delay" command. The code could not respond to any Bluetooth signal

while continuing to count the delays, causing it to lock into a given light rotation. Fixing this issue required rewriting most of the code. I replaced the “delay” command with a system which resets every 12 seconds. In 12 seconds, a full rotation of all the lights elapses. The new system checks the number of seconds that have passed every second, and changes which lights are on and off based on how many seconds have passed in the rotation (for example, if 4 seconds have passed, turn off the green light and turn on the yellow light). Once I finalized the new code, the model could respond to bluetooth connections multiple times without freezing and rotate between 2 different intersecting traffic lights.

Testing Process

The goal of this experiment was to test the accuracy of the Bluetooth connections by seeing how far away a phone would connect and disconnect to the traffic light model in 10 trials. I began the testing process by going outside. At first, I measured 60 feet in a straight line, putting markers each 10 feet to facilitate the measuring process of each trial, and placed the model on the ground at the end of the 60 feet. I tested the distance required for connection by approaching the model along the 60 ft line while holding the phone on Bluetooth auto connect mode (meaning it would automatically connect to the Arduino once it sensed it in range). I recorded the measurement of the point in which the phone connected. I tested the distance for disconnection by turning around once I had reached the model and going back along the same line until the phone disconnected. After 2 trials, I decided that it would be more accurate to measure the distance for disconnection by continuing along the same straight line in the opposite direction rather than turning around once I reached the light, as that is what cars do at traffic lights. I measured an additional 60 feet continuing straight along the line from where I had placed the

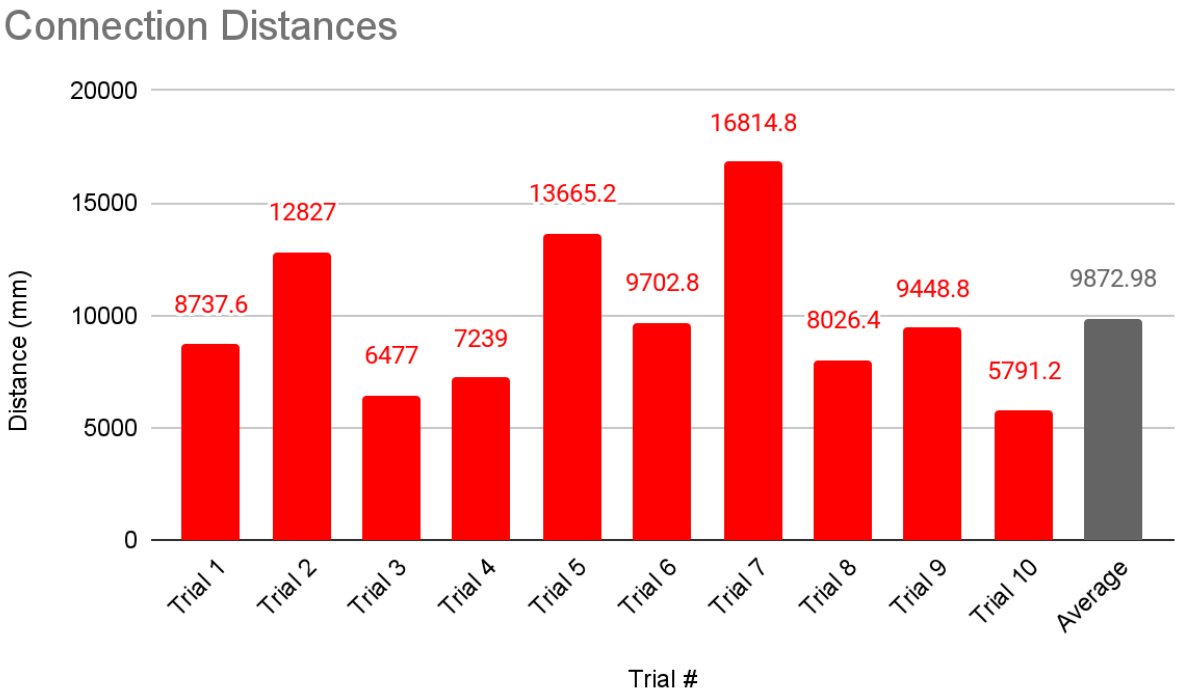
model and restarted my trials. I also noticed that the speed at which I was walking affected the distance of the connections, because walking slower would allow the system more time to process that there was a device in range. For the new set of trials, I synced my steps with a metronome beating at 70 bpm in order to keep a constant speed for every trial and make the measurements more accurate. For each trial, I started at the very beginning of the line (with the Bluetooth disconnected), and walked along the line at the metronome's pace holding the phone until my friend indicated that the lights had turned on. I stopped at the point where the connection occurred, and measured the distance from the model. Once having recorded the connection distance, I continued walking straight until my friend indicated that the lights had turned off. I then recorded the distance from the model to the new spot for disconnection. I repeated this process for 10 trials.

Results and Data

On average, the distance at which the Bluetooth initially connected was 9872.98 mm, while the average disconnect distance was higher, at 13345.16 mm. The connection distances ranged from 5791.2 mm to 16814.8 mm, an overall 11023.6 mm in range. The disconnection distances ranged from 10160 mm to 16814.4 mm, with a smaller overall range of 6654.4 mm.

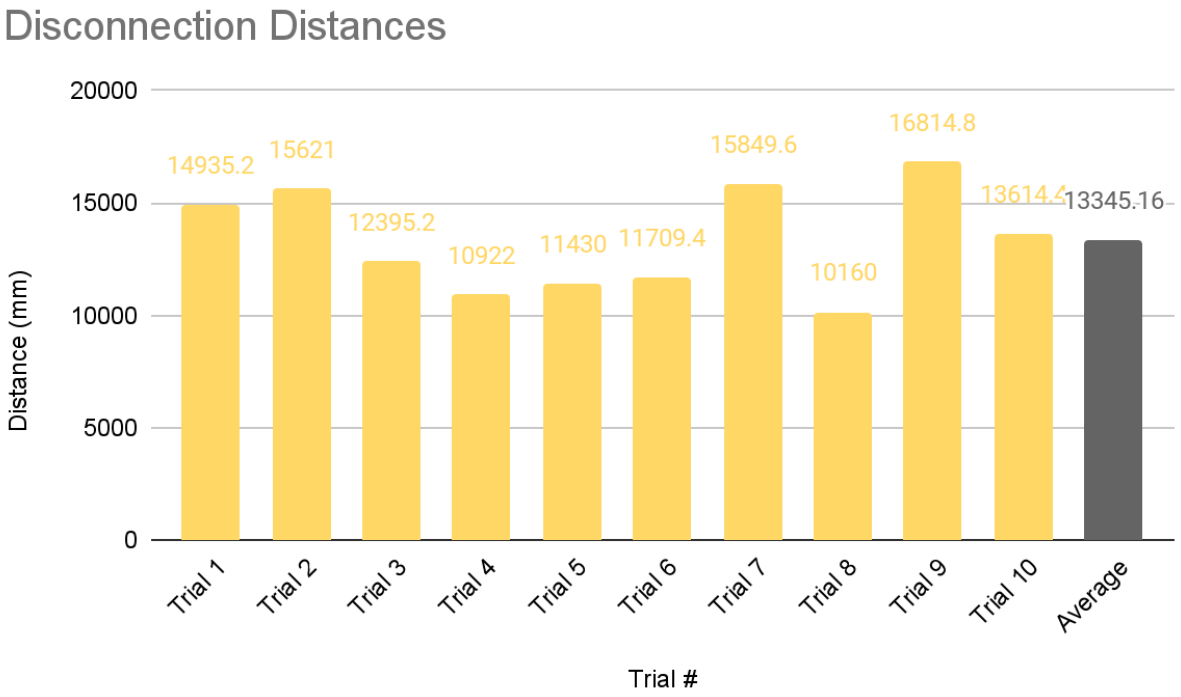
Distance of Each Bluetooth Connection

Connections	
Trial #	**Distance (mm)**
Trial 1	8737.6
Trial 2	12827
Trial 3	6477
Trial 4	7239
Trial 5	13665.2
Trial 6	9702.8
Trial 7	16814.8
Trial 8	8026.4
Trial 9	9448.8
Trial 10	5791.2
Average	*9872.98*
Standard Deviation	*3517.2*


Connection Distances
20000
15000
10000
5000
0
Distance (mm)
8737.6
12827
6477
7239
13665.2
9702.8
16814.8
8026.4
9448.8
5791.2
9872.98
Trial 1
Trial 2
Trial 3
Trial 4
Trial 5
Trial 6
Trial 7
Trial 8
Trial 9
Trial 10
Average
Trial #

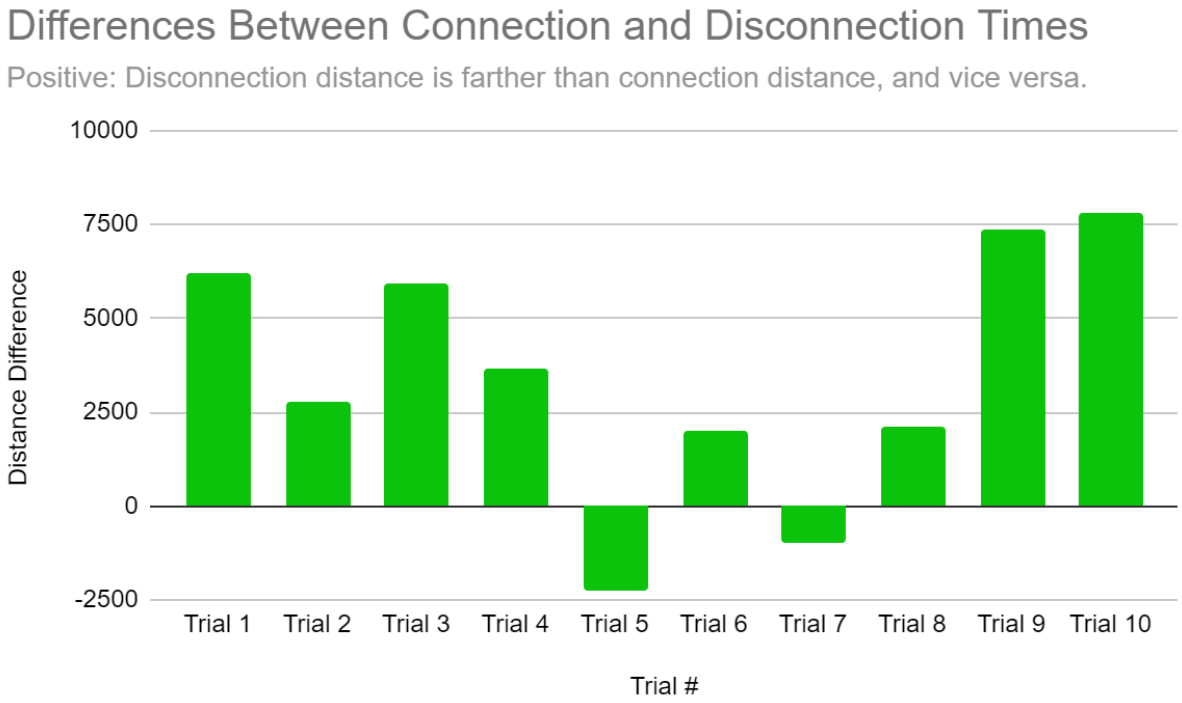
Distance of Each Bluetooth Disconnection

Disconnections	
Trial #	**Distance (mm)**
Trial 1	14935.2
Trial 2	15621
Trial 3	12395.2
Trial 4	10922
Trial 5	11430
Trial 6	11709.4
Trial 7	15849.6
Trial 8	10160
Trial 9	16814.8
Trial 10	13614.4
Average	*13345.16*
Standard Deviation	*2341.7*


Disconnection Distances
20000
15000
10000
5000
0
Distance (mm)
14935.2
15621
12395.2
10922
11430
11709.4
15849.6
10160
16814.8
13614.4
13345.16
Trial 1
Trial 2
Trial 3
Trial 4
Trial 5
Trial 6
Trial 7
Trial 8
Trial 9
Trial 10
Average
Trial #

Difference Between Bluetooth Connection and Disconnection Times

Differences	
Trial #	**Distance (mm)**
Trial 1	6197.6
Trial 2	2794
Trial 3	5918.2
Trial 4	3683
Trial 5	-2235.2
Trial 6	2006.6
Trial 7	-965.2
Trial 8	2133.6
Trial 9	7366
Trial 10	7823.2


Differences Between Connection and Disconnection Times
Positive: Disconnection distance is farther than connection distance, and vice versa.
10000
7500
5000
2500
0
-2500
Distance Difference
Trial 1
Trial 2
Trial 3
Trial 4
Trial 5
Trial 6
Trial 7
Trial 8
Trial 9
Trial 10
Trial #

Calculations

Equation for average distance: $(D_1 + D_2 + D_3 + D_4 + D_5 + D_6 + D_7 + D_8 + D_9 + D_{10}) / 10$

Example: Finding average for connection distances - Variables:

$D_1 = 8737.6$

$D_2 = 12827$

$D_3 = 6477$

$D_4 = 7393$

$D_5 = 13665.2$

$D_6 = 9702.8$

$D_7 = 16814.8$

$D_8 = 8026.4$

$D_9 = 9448.8$

$D_{10} = 5791.2$

Equation:

(8737.6 + 12827 + 6477 + 7393 + 13665.2 + 9702.8 + 16814.8 + 8026.4 + 9448.8 + 5791.2) / 10

= 9872.98

Standard deviation formula (calculated using the Google Sheets STDEV function):

$$\sigma = \sqrt{\frac{\sum(x_i - \mu)^2}{N}}$$

Speed

1. Measure Stride: Amount of steps in 10 ft / 10
2. Multiply Stride by 70 to get distance (ft) traveled in one minute
3. Convert speed from feet/minute to meters/sec

Discussion and Conclusions

The goal of this experiment was to create a model of a traffic light that turns on and alternates between lights when it senses a Bluetooth signal from a phone (representing a car) approaching, and turns off when no signals (cars) can be sensed in range.

Because the distances varied in each trial, an exact distance of connection and disconnection cannot be determined. The trials show that on average, the phone connects around 10 meters away from the traffic light, and disconnects at around 13 meters. On average, the disconnection distances were closer to each other than the connection ones. The connection distance was shorter than the disconnection distance in every trial except for trials 5 and 7. Because of these outliers, it cannot be counted on that the disconnection distance will always be longer. The fact that the disconnections usually take slightly longer is not important, as only the distance at which the light and car connect matters safety wise. The standard deviation for the connection distances was about 3.5 meters, while the standard deviation for the disconnection distances was about 2.3 meters. Ideally, there would have been a reliable distance at which the connection always occurred, and the standard deviation would be lower or zero. A fast moving car would likely require the traffic light to turn on and be seen from a greater distance in order to avoid accidents. However, this is only a small model and a real traffic light would probably have more powerful sensors than the Arduino Wifi board. BLE has a range of up to 100 meters, so it is likely possible to modify this model in order for it to sense cars from greater range and be implemented in a safe way.

The goal was met by the final model. The phone connected and disconnected to the model in every trial, meaning that based on the 10 trials of this experiment, there is a 0% chance of the traffic light not sensing a nearby car (which could cause an accident). The lights always

turned on when connected to Bluetooth and off when disconnected. Also, when the lights were on, they were able to complete the planned 12 second rotation. Therefore, the code was effective in connecting the lights to the Bluetooth signal and implementing the correct light rotation for traffic at an intersection.

Occasionally, while the lights are flashing the rotation, they freeze into a single pattern. This glitch could be caused by a variety of factors: the code might have a loophole that it sometimes falls into or there could be flaws in the assembly of the circuit. This problem occurs more often when the circuit is connected to the Android phone than to an iPhone. I used an android rather than an iPhone in the trials because iPhones do not have an "auto-connect" option for BLE connections, and this feature is necessary for the trials to work. The circuit, driven by the code, might respond differently based on the Bluetooth configurations of the device that is connected to.

The accuracy of the measurements and results could have been affected by various aspects. The speed at which the phone moves influences the distance at which the phone disconnects and connects to the model. If I had chosen a slower speed to walk at during the trials, the connections and disconnections would likely have occurred from farther distances, because the model would have had more time to sense and connect to the Bluetooth. Likewise, if I had chosen a faster speed, the connections and disconnections would probably have occurred closer to the model, as it would have taken the model longer to process the Bluetooth connection. Additionally, the model was on the ground in the trials, while I held the phone in my hands. As a result, the model and phone were not at the same height. This difference in heights also could have affected the distance at which the phone sensed the model. Also, the Android phone that was used in the experiment was an older model. The connections might have been faster and

more accurate with a newer device. The final possible source of inaccuracy in the testing process was the space in which it was held. Part of the 12 foot line along which I walked in the trials was in the space between two houses. These houses could have interfered with the Bluetooth connections by blocking or absorbing signals. Also, in a real traffic light situation, the light would not be bordered by houses very closely.

In order to improve the performance of my current project, a number of different materials could be used to obtain stronger Bluetooth connections. The Arduino board can be replaced with a device with a greater BLE range, so as to sense connections from farther away. A more powerful device could also be used as the Bluetooth transmitter. These changes would allow the traffic light to work in a longer range. The model could also be reproduced on a larger scale so as to actually be applicable to real traffic lights.

Works Cited

Anagnostopoulos, Theodoros, et al. "Cyclist-aware traffic lights through distributed smartphone sensing." *Pervasive and Mobile Computing*, vol. 31, 2016, pp. 22-36.

Anonymous, n.d., Conventional Vs LED Traffic Signals; Operational Characteristics and Economic Feasibility Available from website: https://2.imimg.com/data2/SB/JE/MY-/trafficlights.pdf

"Arduino Tech Specs." *Arduino Uno Rev3*, Arduino, store-usa.arduino.cc/products/arduino-uno-rev3. Accessed 10 Nov. 2022.

Austin, Patrick Lucas. "Want to Fix Road Congestion? Try Smarter Traffic Lights." *Time*, TIME USA, 21 Jan. 2019, time.com/5502192/smart-traffic-lights-ai/. Accessed 26 Oct. 2022.

"Bluetooth Technology Overview." *Bluetooth*, Bluetooth SIG, www.bluetooth.com/learn-about-bluetooth/tech-overview/. Accessed 23 Feb. 2023.

Hillhouse, Grady. "How Do Traffic Signals Work?" Edited by Wesley Crump. *Practical Engineering*, 14 May 2019, practical.engineering/blog/2019/5/11/how-do-traffic-signals-work. Accessed 22 Feb. 2023.

"LED traffic signals save money, time and energy." *Fama Traffic*, Yangzhou Fama Intelligent Equipment CO.,Ltd, 8 May 2020, www.ledtrafficlight.cn/news_view-46.html. Accessed 26 Oct. 2022.

Raikar, Sanat Pai. "How Does Bluetooth Work?" *Encyclopaedia Britannica Online*, 4 Mar. 2022, www.britannica.com/story/how-does-bluetooth-work. Accessed 22 Feb. 2023.

Smith, Stephen F., et al. "SURTRAC: Scalable Urban Traffic Control." *TRB 2013 Annual Meeting*, e-book ed., 2013. pdf.

Taylor, Daniel. "Morning Start: Do you know how much of your life is spent waiting at red lights?" *Keremeos Review*, 2 Apr. 2020. *Keremeos Review*, www.keremeosreview.com/news/morning-start-do-you-know-how-much-of-your-life-is-spent-waiting-at-red-lights/. Accessed 26 Oct. 2022.

Teja, Ravi. "Wireless Communication: Introduction, Types and Applications." *Electronics Hub*, Electronicshub.org, 3 Apr. 2021, www.electronicshub.org/wireless-communication-introduction-types-applications/. Accessed 10 Nov. 2022.

United States Environmental Protection Agency. 11 July 2022, www.epa.gov/energy/about-us-electricity-system-and-its-impact-environment. Accessed 2 Nov. 2022.

Webber, Michael, and Joshua Rhodes. "The Solution to America's Energy Waste Problem is Electrification." *UT News*, The University of Texas at Austin, 19 Dec. 2017, news.utexas.edu/2017/12/19/the-solution-to-americas-energy-waste-problem/#:~:text=Incredible%20as%20it%20may%20sound,energy%20becomes%20useless%20waste%20heat. Accessed 27 Feb. 2023.

Wonderopolis. "How Does a Traffic Light Work?" *Wonder of the Day*. *Wonderopolis*, www.wonderopolis.org/wonder/how-does-a-traffic-light-work. Accessed 26 Oct. 2022.